

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sqlalchemy import create_engine
import os
from dotenv import load_dotenv

load_dotenv(dotenv_path="../../backend/.env")

DATABASE_URL = os.getenv("DATABASE_URL")
engine = create_engine(DATABASE_URL)

# for testing
query = "SELECT * FROM students LIMIT 5;"
df_students = pd.read_sql(query, engine)

df_students.head()
```

```
In [10]: # I'll fetch the grades data by joining the relevant tables
query_grades = """
SELECT g.score, g.is_absent, e.type, m.name as module_name
FROM grades g
JOIN evaluations e ON g.evaluation_id = e.id
JOIN cohort_modules cm ON e.cohort_module_id = cm.id
JOIN modules m ON cm.module_id = m.id;
"""
df_grades = pd.read_sql(query_grades, engine)

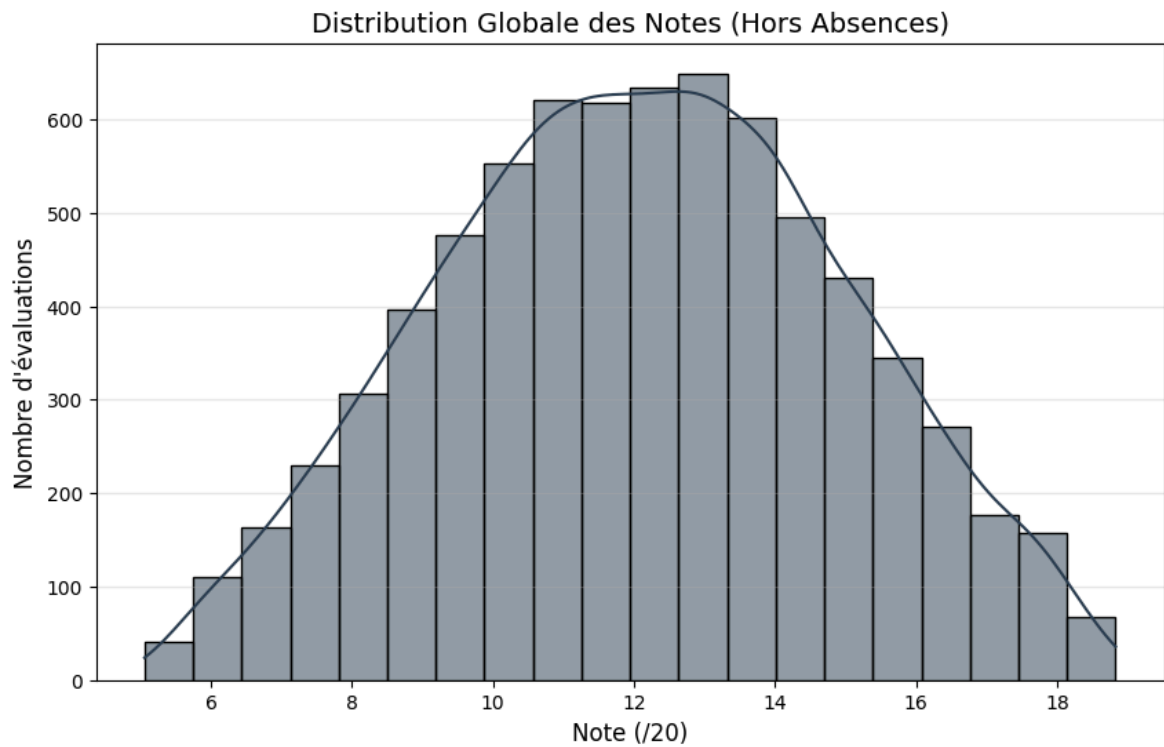
# I'll display a basic statistics (Mean, Min, Max, etc.)
print("Statistiques Descriptives des Notes :")
display(df_grades['score'].describe())

# for visualizion of the distribution of all grades
plt.figure(figsize=(10, 6))
sns.histplot(data=df_grades[df_grades['is_absent'] == False], x='score',
plt.title('Distribution Globale des Notes (Hors Absences)', fontsize=14)
plt.xlabel('Note (/20)', fontsize=12)
plt.ylabel('Nombre d\'évaluations', fontsize=12)
plt.grid(axis='y', alpha=0.3)
plt.show()
```

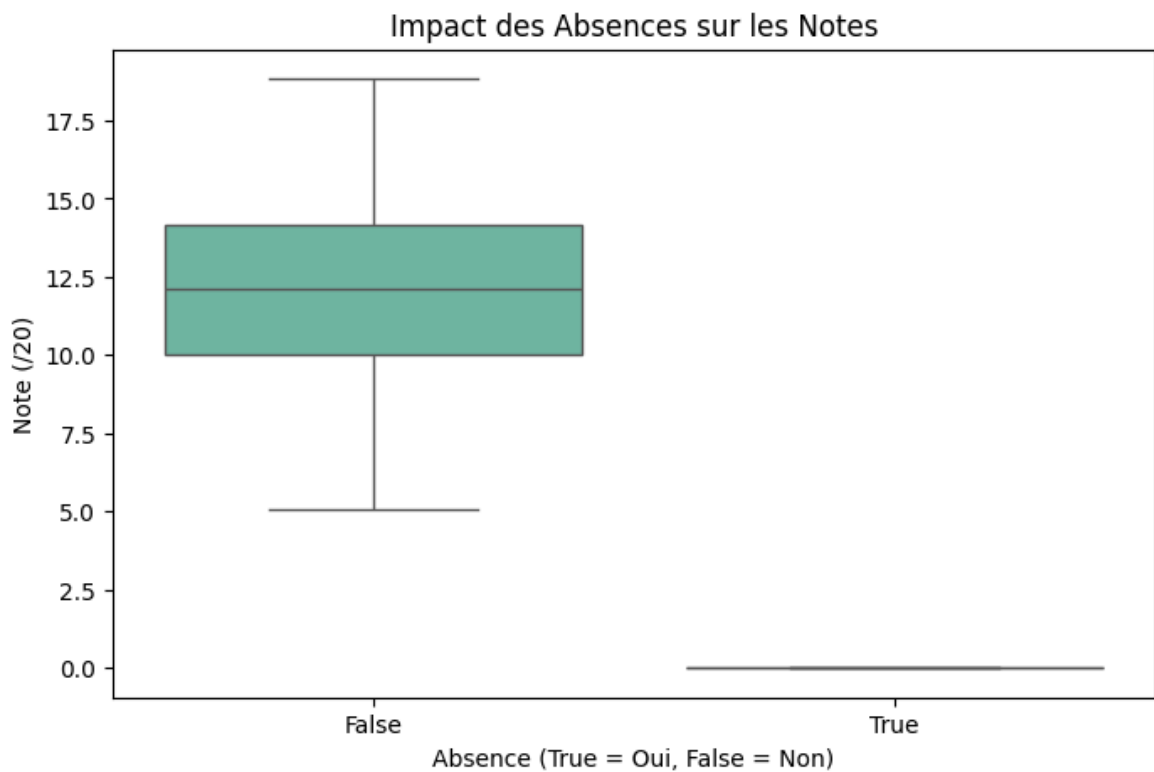
Statistiques Descriptives des Notes :

count	8000.000000
mean	11.097155
std	4.315074
min	0.000000
25%	9.350000
50%	11.740000
75%	13.950000
max	18.830000

Name: score, dtype: float64



```
In [13]: # Visualizing the impact of absences on student scores to detect correlat
plt.figure(figsize=(8, 5))
sns.boxplot(data=df_grades, x='is_absent', y='score', hue='is_absent', pa
plt.title('Impact des Absences sur les Notes')
plt.xlabel('Absence (True = Oui, False = Non)')
plt.ylabel('Note (/20)')
plt.show()
```



```
In [14]: # to get the overall average and the number of absences per student
query_students_risk = """
SELECT
    s.student_id_number,
```

```

        u.first_name,
        u.last_name,
        AVG(g.score) as moyenne_generale,
        SUM(CASE WHEN g.is_absent THEN 1 ELSE 0 END) as total_absences
FROM students s
JOIN users u ON s.user_id = u.id
JOIN grades g ON s.id = g.student_id
GROUP BY s.student_id_number, u.first_name, u.last_name;
"""
df_risk = pd.read_sql(query_students_risk, engine)

seuil_moyenne = 10.0
seuil_absence = 3

etudiants_a_risque = df_risk[
    (df_risk['moyenne_generale'] < seuil_moyenne) |
    (df_risk['total_absences'] > seuil_absence)
].copy()

etudiants_a_risque['moyenne_generale'] = etudiants_a_risque['moyenne_gene

print(f" Alerte : {len(etudiants_a_risque)} étudiants à risque identifiés
display(etudiants_a_risque.sort_values(by='moyenne_generale').head(10))

```

Alerte : 368 étudiants à risque identifiés !

	student_id_number	first_name	last_name	moyenne_generale	total_absences
993	STU-100372	Amélie	Garnier	4.76	4
190	STU-100970	Raymond	Collin	5.26	2
692	STU-100902	Gabrielle	Hoareau	5.27	4
705	STU-100116	Guillaume	Lejeune	5.40	2
762	STU-100778	Arthur	Lebreton	5.50	3
194	STU-100883	Éléonore	Morvan	5.52	3
773	STU-100769	Jacqueline	Moreno	5.57	4
563	STU-100507	Matthieu	Imbert	5.59	2
156	STU-100036	Matthieu	Tanguy	5.92	4
577	STU-100827	Guillaume	Vasseur	5.99	3

```

In [ ]: # --- EVALUATION DU MODÈLE PRÉDICTIF (BONUS A) ---
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sqlalchemy import create_engine
import os
from dotenv import load_dotenv

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score

load_dotenv(dotenv_path="../backend/.env")

```

```

engine = create_engine(os.getenv("DATABASE_URL"))

query = """
SELECT
    s.id as student_id,
    AVG(g.score) as avg_grade,
    SUM(CASE WHEN g.is_absent THEN 1 ELSE 0 END) as total_absences
FROM students s
JOIN grades g ON s.id = g.student_id
GROUP BY s.id;
"""

df_ml = pd.read_sql(query, engine).fillna(0)

df_ml['is_at_risk'] = (df_ml['avg_grade'] < 10.0).astype(int)

X = df_ml[['avg_grade', 'total_absences']]
y = df_ml['is_at_risk']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

print("---- CLASSIFICATION REPORT ----")
print(classification_report(y_test, y_pred))

print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred, zero_division=0):.4f}")
print(f"Recall : {recall_score(y_test, y_pred, zero_division=0):.4f}")
print(f"F1-Score : {f1_score(y_test, y_pred, zero_division=0):.4f}")

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Stable (',
plt.title('Matrice de Confusion - Prédiction du Risque')
plt.xlabel('Prédiction')
plt.ylabel('Réalité')
plt.show()

```

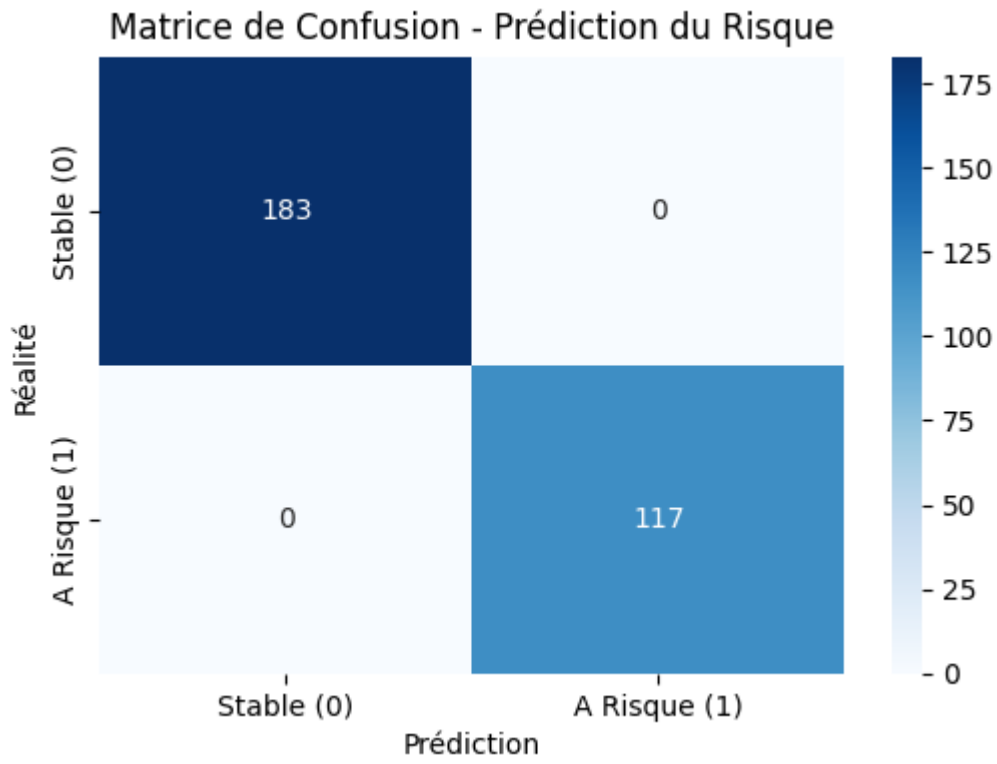
---- CLASSIFICATION REPORT ----

	precision	recall	f1-score	support
0	1.00	1.00	1.00	183
1	1.00	1.00	1.00	117
accuracy			1.00	300
macro avg	1.00	1.00	1.00	300
weighted avg	1.00	1.00	1.00	300

```

Accuracy : 1.0000
Precision : 1.0000
Recall : 1.0000
F1-Score : 1.0000

```



In []: